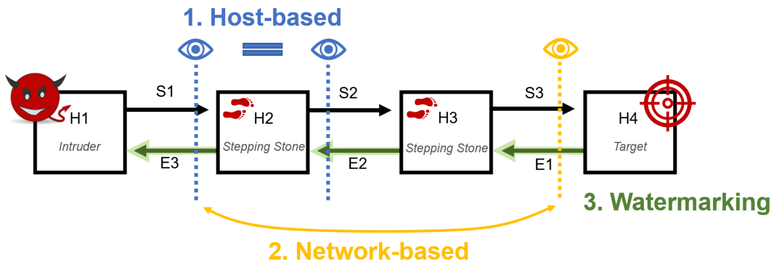
Modeling Stepping-Stone Flows using Stochastic Random Walks and Black Box Neural Networks

Luke Kurlandski (lk3591@g.rit.edu), Nate Matthews (njm3308@g.rit.edu), Nicholas Hopper (hoppernj@umn.edu), and Matthew Wright (mkwics@g.rit.edu)

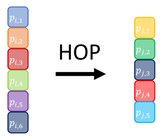
Stepping-Stone Intrusion Defense

- Active SSID via watermarking is a promising defense against intrusions.

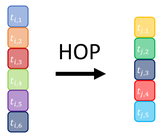


Modeling Flow Transformations

- Modeling the transformation of a flows through hops is a precursor to deep learning-based watermarking systems such as FINN.



- There is no one-to-one correspondence between packets in ingress and egress flows.
- This property breaks the simple Laplacian-based model used for watermarking in FINN.



- Our goal is to develop an enhanced model of flow transformations for SSI interpacket delays.
- Concretely, we aim to develop some model M such that $M(t_i) \cong t_j$ where t_i and t_j are ingress and egress interpacket delays in an SSI chain.

A Statistical Model of Interpacket Delays

- We use a generalized normal distribution to model the interpacket delays.

Model	NLL	μ	s	β
$\mathcal{G}$	-5.89 +/- 1.83	0.00	0.00	0.27 +/- 0.13
$\mathcal{L}$	-2.94 +/- 0.01	0.00	0.01	NA
$\mathcal{N}$	-0.99 +/- 0.08	0.01	0.09	NA

We then fit a model on our entire dataset:
$\mu = 0.0000000$
$s = 1.0096913 \times 10^{-6}$
$\beta = 0.25405909$

A Stochastic Model of Flow Lengths

- We propose a stochastic model of the number of packets in an egress flow based on the number of packets in an ingress flow.
- We assume three possible operations that each packet can undergo: a drop, a chop, and a noop, which occur with probabilities p_d, p_c, p_n.



- Then final length of a flow can then be computed using Algorithm 1

Algorithm 1 Stochastic New Flow Length Calculation

Require: l_i, p_d, p_c, p_n
  $l_f \leftarrow 0$
  $n \leftarrow 0$
  while $i < l_i$ **do**
    $n \leftarrow n + 1$
    $r \leftarrow \mathcal{U}(0, 1)$
    if $r < p_d$ **then** ▷ Drop
      $l_f \leftarrow l_f + 0$
      $i \leftarrow i + 1$
    else if $r < p_d + p_c$ **then** ▷ Chop
      $l_f \leftarrow l_f + 1$
      $i \leftarrow i + 0$
    else ▷ Noop
      $l_f \leftarrow l_f + 1$
      $i \leftarrow i + 1$
    end if
  end while
  return l_f, n

- Algorithm 1 is isomorphic to a random walk with particular properties.

$$\mathbb{E}(l_f) = \mathbb{E}(l_i) + \sum_{i=1}^{n} p_c - p_d$$

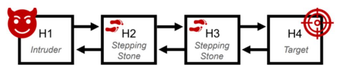
- However, the number of steps in this walk is a nondeterministic function.

$$\mathbb{E}(l_f) = \mathbb{E}(l_i) + \sum_{i=1}^{\mathbb{E}(f(x))} p_c - p_d$$

- If we approximate the function, we can see a clear relationship

$$p_c - p_d = \frac{\mathbb{E}(l_f) - \mathbb{E}(l_i)}{g(x, \Theta)}$$

To attain probabilities p_d, p_c, p_n that describe a dataset:
1) Select a polynomial function g,
2) Fit the learnable parameters Θ,
3) Choose an arbitrary p_c,
4) Find a solution for p_d using g, Θ, $\overline{l_i}$, $\overline{l_f}$
5) Solve for $p_n = 1 - p_d - p_c$.



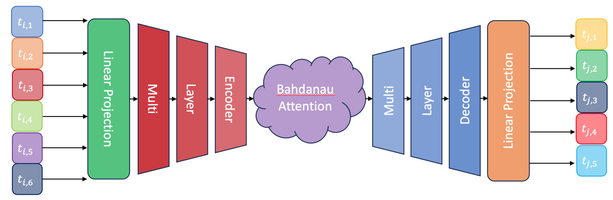
To fit learnable parameters for our dataset (step 2) we
1) Generate a synthetic dataset using Algorithm 1
2) Select the best polynomial model
3) Compare the performance to other ML approaches

Algorithm	R^2	EVAR	MSE	MAE
Linear	0.31	0.31	1.11e+03	1.16e+07
Polynomial	0.91	0.91	3.53e+02	1.52e+06
MLP	0.60	0.60	3.29e+02	6.67e+06
GBDT	0.93	0.93	8.84e+01	1.24e+06
RF	0.92	0.92	4.91e+01	1.29e+06
SVM	0.29	0.30	4.36e+02	1.18e+07

Finally, we solve for the probabilities:
$p_d = 0.51939008$
$p_c = 0.25000000$
$p_n = 0.23060992$

A Neural Model of Flow Transformations

- We propose to use a neural network to learn the transformation end-to-end.
- We use an encoder-decoder sequence-to-sequence architecture. We experiment with RNN, LSTM, GRU, and Transformer blocks.



- We train the model to minimize the error between true and predicted interpacket delays along with the the error between the number of packets.

$$\mathcal{L} = K_1(N - M)^2 + K_2 \sum_{i}^{\min(N,M)} (y_i - \hat{y}_i)^2$$

- We find that this approach exceeds the statistical/stochastic model at IPD prediction, but more work is needed to improve the networks' ability to learn to stop generating interpacket delays and learn the correct target length.

Length	Arch	R^2	NRMSE	NDEV
64	BASE	-0.077	3.810	17.097
	TRN	0.006	3.939	1.513
	GRU	0.054	3.843	1.434
	RNN	0.110	3.728	1.339
	LSTM	0.040	3.871	1.500
128	BASE	-0.050	4.611	21.989
	TRN	0.005	5.346	1.724
	GRU	0.009	5.337	1.555
	RNN	0.028	5.285	1.401
	LSTM	0.006	5.344	1.765
256	BASE	-0.032	5.708	22.724
	TRN	0.005	7.730	1.676
	GRU	0.005	7.728	1.604
	RNN	0.015	7.692	1.668
	LSTM	0.006	7.724	1.559

Length	Arch	R^2	MSE	MAE
64	BASE	-3109.773	5.37e+02	22.263
	TRN	-0.000	7.81e-02	0.006
	GRU	-0.000	7.81e-02	0.006
	RNN	-0.000	7.81e-02	0.006
	LSTM	-0.000	7.81e-02	0.006
128	BASE	-26.467	2.30e+03	46.024
	TRN	-0.039	8.72e+01	1.810
	GRU	-0.039	8.72e+01	1.810
	RNN	-0.039	8.72e+01	1.810
	LSTM	-0.039	8.72e+01	1.810
256	BASE	-2.610	8.78e+03	85.827
	TRN	-0.590	3.84e+03	37.742
	GRU	-0.590	3.84e+03	37.742
	RNN	-0.590	3.84e+03	37.742
	LSTM	-0.590	3.84e+03	37.742

Performance on IPD prediction (left) and length prediction (right)

Future Work

- Modify end-to-end model:
  - Add regression head to directly predict sequence length
  - Add loss term to push output distributions closer together
- Implement an active defense against SSI
  - Insert new hop models into FINN framework
  - Improve backbone FINN framework for SSI traffic

This project is funded by **NCAE-C-003-2022** under award **#H98230-22-1-0302**.